

Prototipo de red de sensores para fusión de conocimiento

Sistemas Empotrados y Ubicuos

Máster Universitario en Ingeniería Informática — Universidad de Jaén

Curso 2025-26



Realizado por Víctor Aguilar, Jaime Cazalla y Javier Dibo

Índice de contenidos

1. Introducción

2. Alternativas de solución y herramientas empleadas

- 2.1 Captura de datos: app propia frente a AWARE
- 2.2 Comunicación: síncrona (HTTP) frente a asíncrona (MQTT)
- 2.3 Almacenamiento: base de datos relacional frente a serie temporal
- 2.4 Visualización: aplicación propia frente a Grafana
- 2.5 Empaquetado: Docker Compose

3. Diseño y desarrollo del prototipo

- 3.1 Arquitectura general
- 3.2 Conjunto de sensores utilizado
- 3.3 Aplicación Android (productor)
- 3.4 API puente (FastAPI)
- 3.5 Diseño de la base de datos: tablas, atributos y tipos
- 3.6 Proceso de recogida de datos y su justificación

4. Experimentación y resultados

- 4.1 Validación funcional del sistema
- 4.2 Descripción del dataset recogido
- 4.3 Visualización y caso de uso: fusión de conocimiento
- 4.4 Ajuste de rendimiento y robustez del pipeline

5. Conclusiones y autoevaluación

Autoevaluación

6. Bibliografía

7. Apéndices

- A. Manual de despliegue del backend
- B. Configuración de la app móvil
- C. Configuración de Grafana

1. Introducción

Vivimos rodeados de dispositivos capaces de medir el mundo que nos rodea. Un teléfono inteligente moderno integra una decena de sensores —acelerómetro, giroscopio, magnetómetro, sensor de luz, de proximidad, de presión, etc.— que generan, de forma continua, un flujo de datos enormemente valioso. El reto no es solo *recoger* esos datos, sino **transmitirlos, almacenarlos y analizarlos** para extraer conocimiento útil, como por ejemplo reconocer la actividad que está realizando el usuario.

Este es precisamente el problema que aborda la práctica: construir un **prototipo de red de sensores** (en inglés, *Wireless Sensor Network*, WSN) que cubra las tres etapas clásicas de un sistema de este tipo:

1. **Recogida** de datos a partir de los sensores de un dispositivo móvil.
2. **Transmisión y almacenamiento** de dichos datos en un servidor con una base de datos.
3. **Recuperación, visualización y análisis** de la información almacenada, prestando especial atención al problema de la **fusión de conocimiento**: combinar datos de distinta naturaleza y origen, alineados en el tiempo, para inferir información de más alto nivel.

El presente documento describe el problema, las alternativas de solución estudiadas, la justificación de las herramientas elegidas, el diseño y desarrollo del prototipo, y los resultados experimentales obtenidos.

2. Alternativas de solución y herramientas empleadas

El prototipo se descompone en cuatro decisiones tecnológicas independientes: cómo **capturar** los datos, cómo **transmitirlos**, dónde **almacenarlos** y cómo **visualizarlos**. Para cada una se valoraron varias alternativas.

2.1 Captura de datos: app propia frente a AWARE

Alternativa	Ventajas	Inconvenientes
AWARE Framework	Solución madura y multiplataforma; muchos sensores listos	Caja negra; poco control sobre el formato; menos valor formativo
App Android propia	Control total del formato y los topics; código propio defendible; aprendizaje	Hay que desarrollarla

Se optó por desarrollar una **aplicación Android propia** (paquete `com.ujd.sensores`). La razón principal es de aprendizaje y control: al ser código propio podemos justificar cada decisión de diseño en la defensa, elegir exactamente el formato del mensaje y la nomenclatura de los topics, y ajustar la frecuencia de muestreo. La app se diseñó, además, para ser **compatible con el resto del sistema sin acoplamiento**: publica por MQTT igual que lo haría cualquier otro nodo de la red.

2.2 Comunicación: síncrona (HTTP) frente a asíncrona (MQTT)

Una WSN se caracteriza por múltiples fuentes que emiten datos de forma continua. Se compararon dos paradigmas:

Alternativa	Modelo	Idoneidad para la WSN
HTTP/REST	Petición-respuesta (síncrono)	Sencillo, pero pesado: cabeceras grandes y una conexión por envío
MQTT	Publicación-suscripción (asíncrono)	Ligero, orientado a streaming, desacopla productor y consumidor

Se eligió **MQTT** por ser el estándar de facto en IoT y el más adecuado para la transmisión **asíncrona** y de alta frecuencia de múltiples sensores. Sus ventajas decisivas son:

- **Ligereza**: la cabecera fija de un mensaje MQTT es de apenas 2 bytes, frente a las decenas de bytes de cabeceras HTTP. Es ideal para dispositivos con energía y ancho de banda limitados.
- **Desacoplo productor/consumidor**: el móvil (publicador) no necesita conocer ni esperar al servidor; publica en un *topic* y el *broker* se encarga del reparto. Se pueden añadir nuevos consumidores (otra base de datos, un panel, etc.) sin tocar al emisor.
- **Modelo de topics jerárquico**: encaja de forma natural con una red de sensores (`sensors/<tipo>`), y permite suscripciones con comodines (`#`).

El **formato del paquete** es **JSON** (texto), por su legibilidad y porque se autodescribe; el tamaño típico de cada mensaje es de ~80–120 bytes. La calidad de servicio elegida es **QoS 0** (*at most once*): para un flujo de alta frecuencia, la pérdida ocasional de una muestra es irrelevante y evitamos la sobrecarga de confirmaciones.

Como **broker** se utiliza **Eclipse Mosquitto**, ligero, estándar y trivial de desplegar en contenedor.

2.3 Almacenamiento: base de datos relacional frente a serie temporal

Alternativa	Modelo	Idoneidad
MySQL / PostgreSQL	Relacional	Requiere esquema rígido; menos óptimo para datos indexados por tiempo
InfluxDB 2	Serie temporal	Diseñado para datos <code>(tiempo, valor)</code> ; retención y <i>downsampling</i> nativos

Los datos de sensores son, por naturaleza, **series temporales**: secuencias de valores indexadas por el instante de medida. Por ello se eligió **InfluxDB 2**, una base de datos específica para este tipo de datos, que ofrece compresión eficiente, políticas de retención, agregaciones temporales y una integración directa con la herramienta de visualización.

2.4 Visualización: aplicación propia frente a Grafana

Para la etapa de análisis se eligió **Grafana** en lugar de desarrollar una aplicación web a medida. Grafana se conecta directamente a InfluxDB, permite construir paneles en **tiempo real** sin escribir código de *frontend*, y es la herramienta estándar de la industria para observabilidad. Esto nos permite concentrar el esfuerzo en el **análisis** (las consultas y la fusión de datos) en vez de en la mecánica de dibujar gráficas.

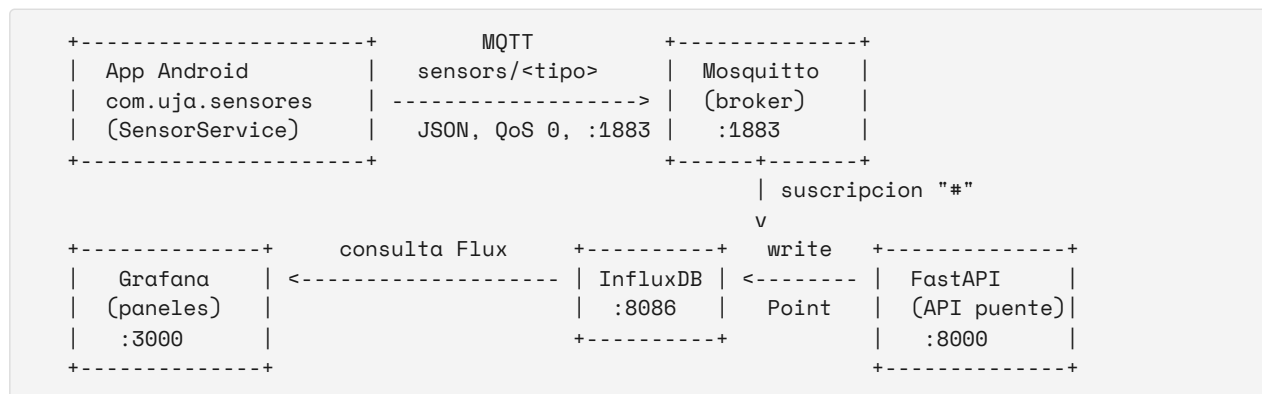
2.5 Empaquetado: Docker Compose

Todo el *backend* (broker, base de datos, API y panel) se orquesta con **Docker Compose**. Esto garantiza un despliegue **reproducible** con un único comando, aísla las dependencias y facilita la corrección y la defensa del trabajo en cualquier máquina.

3. Diseño y desarrollo del prototipo

3.1 Arquitectura general

El sistema sigue el flujo clásico de una red de sensores: un nodo emisor (el móvil) publica las lecturas, que viajan a través del broker hasta un servidor que las almacena y, finalmente, se visualizan.



El *backend* se compone de **cuatro contenedores** definidos en `docker_compose.yaml`: Mosquitto (:1883), InfluxDB (:8086), Grafana (:3000) y la API en FastAPI (:8000).

3.2 Conjunto de sensores utilizado

La aplicación enumera dinámicamente **todos los sensores** disponibles en el dispositivo (`SensorManager.getSensorList(TYPE_ALL)`) y permite al usuario activar los que desee. El conjunto concreto depende del modelo de teléfono; la siguiente tabla describe los sensores típicos empleados y sus características:

Sensor	Magnitud física	Componentes (campos)	Unidad	Rango típico
Acelerómetro	Aceleración (incl. gravedad)	x, y, z	m/s ²	±20 (≈2 g)
Giroscopio	Velocidad angular	x, y, z	rad/s	±10
Magnetómetro	Campo magnético	x, y, z	μT	±100
Aceleración lineal	Aceleración sin gravedad	x, y, z	m/s ²	±20
Gravedad	Vector de gravedad	x, y, z	m/s ²	±9.81
Vector de rotación	Orientación (cuaternión)	x, y, z (, w)	adimensional	[-1, 1]
Luz ambiental	Iluminancia	valor	lux	0 – 40 000
Proximidad	Distancia a objeto	valor	cm	0 – 5 (binario en muchos)
Presión (barómetro)	Presión atmosférica	valor	hPa	300 – 1100
Podómetro	Pasos acumulados	valor	conteo	≥ 0

Los sensores **inerciales** (acelerómetro, giroscopio) son los más relevantes para el caso de uso de reconocimiento de actividad; los **ambientales** (luz, presión) aportan contexto adicional.

3.3 Aplicación Android (productor)

La app, escrita en **Kotlin**, consta de tres componentes:

- **MainActivity** : interfaz de usuario. Muestra un campo para la **IP del servidor** y una lista de **interruptores**, uno por sensor disponible. Al pulsar *Iniciar transmisión* valida la entrada y arranca el servicio en segundo plano. Solicita el permiso `ACTIVITY_RECOGNITION`.
- **SensorService** : un **foreground service** (servicio en primer plano con notificación persistente) que sobrevive aunque la app pase a segundo plano. Adquiere un `WakeLock` parcial para que la CPU no se duerma, registra los *listeners* de los sensores seleccionados y, por cada lectura, construye el mensaje y lo publica.
- **MqttManager** : encapsula el cliente **Paho MQTT**. Gestiona la conexión (`tcp://<ip>:1883` , sesión limpia, *timeout* 10 s) y la publicación de mensajes.

Nomenclatura de topics. Cada lectura se publica en un topic derivado del **tipo estándar** del sensor, no de su nombre comercial. A partir de `sensor.stringType` (p. ej. `android.sensor.accelerometer`) se obtiene un nombre limpio y se compone el topic:

```
private fun topicForSensor(sensor: Sensor): String {
    val clean = (sensor.stringType ?: "type_{$sensor.type}")
        .substringAfterLast('.') // "android.sensor.accelerometer" -> "accelerometer"
        .replace(' ', '_')
        .lowercase()
    return "sensors/$clean" // -> "sensors/accelerometer"
}
```

Formato del mensaje. Cada muestra es un objeto JSON con los valores del sensor, la precisión y la marca de tiempo en milisegundos:

```
{ "value_0": 0.12, "value_1": 0.05, "value_2": 9.79, "accuracy": 3, "timestamp": 1700000000000 }
```

Para los sensores de varios ejes, `value_0`, `value_1` y `value_2` corresponden a las componentes x, y, z respectivamente; los sensores de un solo valor (luz, presión...) solo incluyen `value_0`.

3.4 API puente (FastAPI)

El componente que conecta el mundo MQTT con la base de datos es una pequeña API en **FastAPI** (`api/main.py`). Su ciclo de vida se gestiona con un *lifespan handler*: al arrancar, conecta con el broker y lanza el bucle MQTT en un hilo de fondo; al apagarse, cierra ordenadamente las conexiones. Se **suscribe al topic** # (todos), de modo que captura cualquier sensor sin configuración previa, y por cada mensaje:

1. Decodifica el JSON recibido.
2. Crea un *Point* de InfluxDB cuyo **measurement es el propio topic** (p. ej. `sensors/accelerometer`).
3. Añade cada par clave/valor del JSON como un **campo** del punto, forzando a `float` las lecturas `value_*` para que su tipo no varíe entre mensajes (ver más abajo).
4. **Sella el punto con una marca de tiempo de alta resolución** (nanosegundos) del reloj del servidor.
5. Lo entrega al **escritor por lotes**, que acumula los puntos y los vuelca a InfluxDB en bloque.

```
def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode("utf-8"))
    point = Point(msg.topic) # el topic MQTT -> measurement
    for key, value in data.items():
        # value_* -> float (tipo estable); accuracy/timestamp se dejan como están
        point.field(key, float(value) if key.startswith("value") else value)
    point.time(time.time_ns(), write_precision=WritePrecision.NS) # marca única
    write_api.write(bucket=INFLUXDB_BUCKET, org=INFLUXDB_ORG, record=point)
```

Nota de implementación: la biblioteca `paho-mqtt` 2.x exige declarar la versión de la API de *callbacks*. Se fija `CallbackAPIVersion.VERSION1` para mantener las firmas clásicas `on_connect(client, userdata, flags, rc)` y `on_message(client, userdata, msg)`.

El motivo de la escritura **por lotes** y de la **marca de tiempo explícita** —y los problemas que resuelven— se detallan en la sección 4.4.

3.5 Diseño de la base de datos: tablas, atributos y tipos

InfluxDB es una base de datos de **series temporales** y, por tanto, *schemaless*: no se definen tablas por adelantado, sino que se crean implícitamente al escribir. Su modelo de datos se corresponde con el relacional de la forma siguiente:

Concepto InfluxDB	Equivalente relacional	En nuestro sistema
<i>Bucket</i>	Base de datos	<code>sensores_bucket</code>
<i>Measurement</i>	Tabla	Un <i>measurement</i> por topic: <code>sensors/accelerometer</code> , <code>sensors/gyroscope</code> , ...
<i>Tag</i>	Columna indexada (metadato)	(no se usan; ver más abajo)
<i>Field</i>	Columna de valor	<code>value_0</code> , <code>value_1</code> , <code>value_2</code> , <code>accuracy</code> , <code>timestamp</code>
<i>Timestamp</i>	Clave primaria temporal	<code>_time</code> , asignado por el servidor al escribir

Así, cada tipo de sensor genera una "tabla" (measurement) con la siguiente estructura de atributos y tipos:

Measurement `sensors/accelerometer` (análogo para giroscopio, magnetómetro, etc.):

Atributo	Tipo	Descripción
<code>_time</code>	timestamp (ns)	Instante de escritura (clave temporal)
<code>value_0</code>	float	Componente X
<code>value_1</code>	float	Componente Y
<code>value_2</code>	float	Componente Z
<code>accuracy</code>	integer	Precisión reportada por Android (0-3)
<code>timestamp</code>	integer	Marca de tiempo del dispositivo (ms)

Measurement `sensors/light` (sensores de un único valor):

Atributo	Tipo	Descripción
<code>_time</code>	timestamp (ns)	Instante de escritura
<code>value_0</code>	float	Iluminancia (lux)
<code>accuracy</code>	integer	Precisión (0-3)
<code>timestamp</code>	integer	Marca de tiempo del dispositivo (ms)

Sobre el uso de tags. En el diseño actual todos los datos se almacenan como *fields*. Como mejora se podría añadir un *tag device* (para distinguir varios móviles en la red) o un *tag activity* (para etiquetar la actividad en origen). En este prototipo, el etiquetado de la actividad se resuelve por **ventanas temporales** (ver 3.6), lo que evita modificar la app.

3.6 Proceso de recogida de datos y su justificación

La recogida se realiza en **streaming continuo**: mientras el servicio está activo, cada sensor emite a la máxima frecuencia (`SENSOR_DELAY_FASTEST`), lo que maximiza la resolución temporal para detectar patrones de movimiento. El *foreground service* y el *WakeLock* garantizan que la captura no se interrumpa aunque la pantalla se apague.

Para poder **analizar y etiquetar** los datos sin modificar la aplicación, la recogida se organiza por **ventanas temporales etiquetadas**: se registra cada actividad durante un intervalo conocido (p. ej., 5 minutos *parado*, seguidos de 5 minutos *caminando*), anotando las horas de inicio y fin. Posteriormente, en la fase de análisis, basta con filtrar por rango de tiempo en InfluxDB/Grafana para separar cada actividad. Sobre esos mismos datos se aplica, además, una **clasificación automática por umbrales** (sección 4.3) que no requiere etiquetado manual.

4. Experimentación y resultados

4.1 Validación funcional del sistema

Antes de la recogida real se validó el *pipeline* completo de forma aislada: se levantó el *backend* con Docker Compose y se publicaron mensajes de prueba en los topics `sensors/accelerometer`, `sensors/gyroscope` y `sensors/light`. Se comprobó que la API los recibía, los parseaba y los almacenaba correctamente en InfluxDB, donde quedaban disponibles para su consulta (cada *measurement* con sus campos `value_*`, `accuracy` y `timestamp` y los tipos esperados). El sistema, por tanto, funciona de extremo a extremo.

4.2 Descripción del dataset recogido

El dataset se capturó en una única sesión con un teléfono Android retransmitiendo en tiempo real hacia el *backend*. La transmisión usó primero el túnel USB (depuración por ADB) y, una vez verificado el sistema, una **red WiFi compartida** por el propio móvil. La tabla resume sus características:

Característica	Valor
Sensores empleados (nº de <i>measurements</i>)	18 (acelerómetro, giroscopio, aceleración lineal, gravedad, magnetómetro, vectores de rotación, orientación, luz, presión, podómetro, detector de pasos...)
Nº de atributos por muestra	5 en sensores de varios ejes (<code>value_0</code> , <code>value_1</code> , <code>value_2</code> , <code>accuracy</code> , <code>timestamp</code>); 3 en los de un solo valor
Frecuencia de muestreo aproximada	~280 Hz acelerómetro y giroscopio (modo FASTEST); ~140 Hz aceleración lineal; ~50 Hz gravedad y rotación; ~24 Hz magnetómetro; ~6 Hz presión; <1 Hz luz; detectores por evento
Duración de la captura	~37 min (2 197 s)
Nº de casos totales	3 257 417 muestras (≈ 21,2 millones de puntos campo-valor)
Actividades etiquetadas	parado / caminando / corriendo
Metadatos	dispositivo Android; organización <code>ujaen</code> ; <code>bucket</code> <code>sensores_bucket</code> ; fecha 01/06/2026; transporte USB y WiFi; ventanas temporales por actividad

El reparto de muestras por sensor (contando el campo `value_0`) refleja las distintas frecuencias nativas de cada uno:

Measurement	Muestras	Measurement	Muestras
sensors/gyroscope_uncalibrated	622 298	sensors/orientation	53 066
sensors/accelerometer	622 263	sensors/magnetic_field	53 057
sensors/gyroscope	622 253	sensors/magnetic_field_uncalibrated	53 056
sensors/accelerometer_uncalibrated	622 213	sensors/geomagnetic_rotation_vector	53 056
sensors/linear_acceleration	311 159	sensors/pressure	13 264
sensors/gravity	109 667	sensors/flicker	10 611
sensors/game_rotation_vector	109 666	sensors/light	1 510

(Se omiten varios sensores de evento con muy pocas muestras: `step_detector`, `step_counter`, `device_orientation` y `tilt_detector`.) La cifra confirma el orden de magnitud esperado: a ~280 Hz, solo el acelerómetro aporta más de 600 000 muestras en la sesión, y el conjunto supera los 3,2 millones de muestras (21 millones de valores individuales).

4.3 Visualización y caso de uso: fusión de conocimiento

El caso de uso que ilustra la **fusión de conocimiento** es el **reconocimiento de actividad**: distinguir si el usuario está **parado**, **caminando** o **corriendo** combinando varios sensores alineados sobre su eje temporal común.

Magnitud de la aceleración. La señal más informativa es la magnitud del vector de aceleración, $\alpha = \sqrt{x^2 + y^2 + z^2}$, que fusiona los tres ejes del acelerómetro en un único valor: en reposo se mantiene cerca de 9,8 m/s² (solo gravedad) y prácticamente constante, mientras que al caminar o correr oscila de forma marcada y periódica. La consulta **Flux** que la calcula para un panel de Grafana es:

```
import "math"

from(bucket: "sensores_bucket")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "sensors/accelerometer")
  |> filter(fn: (r) => r._field == "value_0" or r._field == "value_1" or r._field == "value_2")
  |> pivot(rowKey: ["_time"], columnKey: ["_field"], valueColumn: "_value")
  |> filter(fn: (r) => exists r.value_0 and exists r.value_1 and exists r.value_2)
  |> map(fn: (r) => ({ _time: r._time,
    _value: math.sqrt(x: r.value_0 * r.value_0
      + r.value_1 * r.value_1
      + r.value_2 * r.value_2) })))
```

(El filtro `exists` descarta filas incompletas que el `pivot` puede generar en el borde de la ventana de consulta; el porqué se explica en la sección 4.4.)

Fusión de dos sensores. Si a la intensidad de movimiento del acelerómetro se le superpone la del **giroscopio** (magnitud de la velocidad angular, $\omega = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}$), se aprecia que ambas señales se "encienden" simultáneamente con el movimiento. La figura 1 representa, para toda la captura, la intensidad de movimiento (desviación típica en ventanas de 2 s) de ambos sensores: los picos coinciden casi perfectamente, lo que confirma que combinar dos fuentes independientes sobre el eje temporal común permite **inferir la actividad** con más fiabilidad que cualquiera por separado.

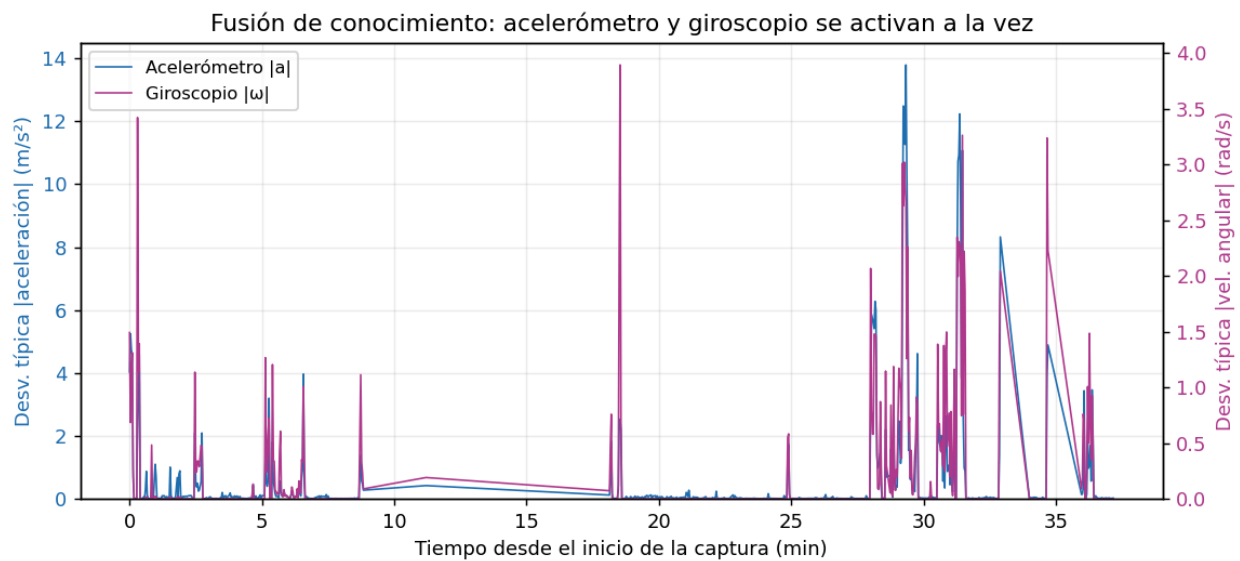


Figura 1. Fusión de conocimiento: la intensidad de movimiento del acelerómetro (azul) y del giroscopio (magenta) aumentan de forma sincronizada. La región de actividad intensa (min 27–36) corresponde a la fase etiquetada de caminar y correr.

Clasificación automática de la actividad. Más allá de la inspección visual, el panel implementa un **clasificador automático por umbrales**. Para cada ventana de 2 s se calcula la **desviación típica** de la magnitud de la aceleración —una medida de cuánto se agita el dispositivo, próxima a 0 en reposo y creciente con la intensidad del movimiento— y se asigna un estado según dos umbrales configurables (`umbral_caminar` y `umbral_correr`, expuestos como variables de Grafana):

```
// ... (magnitud de la aceleración, como arriba)
|> aggregateWindow(every: 2s, fn: stddev, createEmpty: false)
|> map(fn: (r) => ({ _time: r._time,
    _value: if r._value < ${umbral_caminar} then 0.0      // Parado
              else if r._value < ${umbral_correr} then 1.0 // Caminando
              else 2.0 })))                             // Corriendo
```

Esto alimenta dos paneles: un **diagrama de estados temporal** (*state-timeline*) que pinta cada ventana de verde/naranja/rojo, y una **tabla** que acumula el tiempo total en cada estado dentro del rango seleccionado. Tras calibrar los umbrales (0,5 y 4,0 m/s²) realizando físicamente cada actividad, la clasificación sobre la captura completa arrojó:

Estado	Tiempo	Porcentaje
Parado	1 300 s (21:40)	85 %
Caminando	178 s (2:58)	12 %
Corriendo	54 s (0:54)	4 %

La figura 2 muestra la intensidad de movimiento a lo largo de toda la sesión, coloreada según el estado clasificado, junto con el reparto de tiempo. Se distingue con claridad la larga fase de reposo inicial (el teléfono sobre la mesa) y la fase final de actividad etiquetada, en la que el clasificador separa correctamente los tramos de caminar (naranja) de los de correr (rojo).

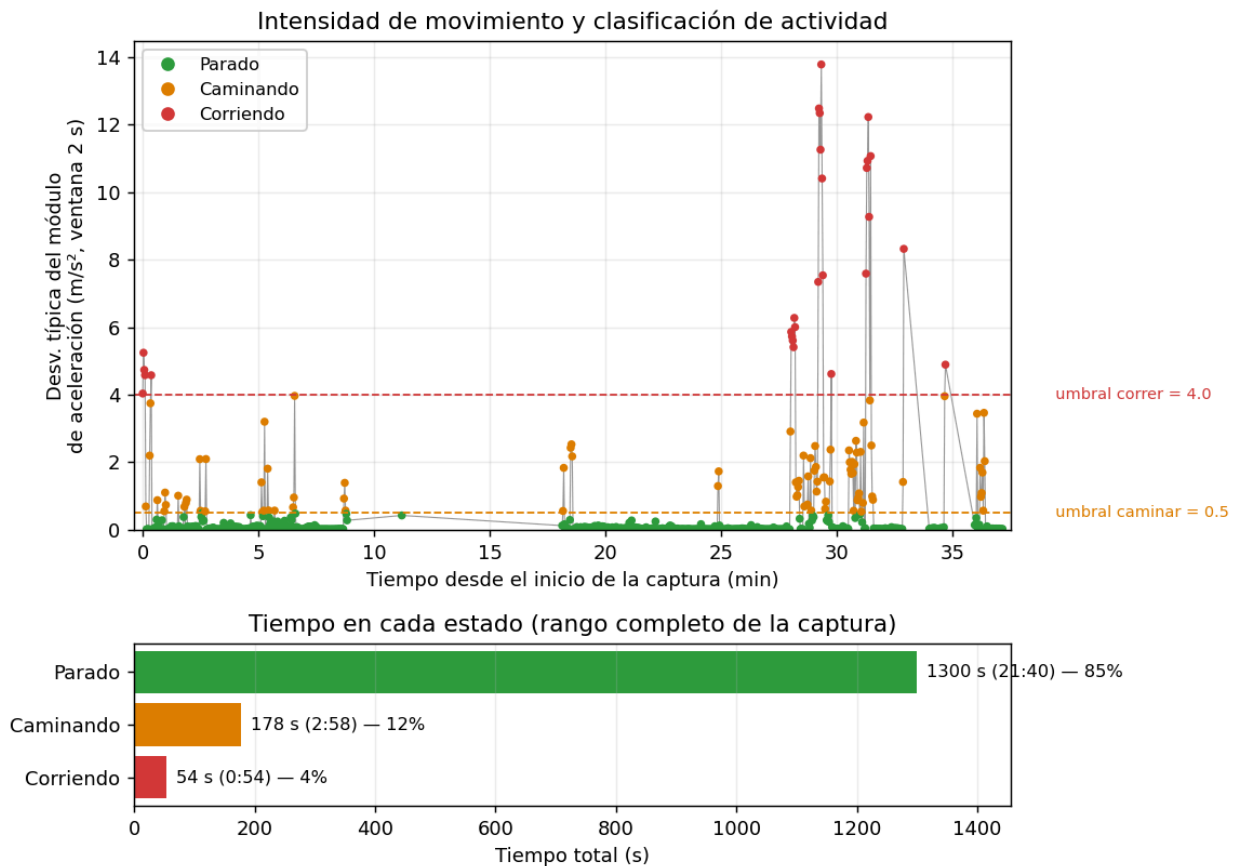


Figura 2. (Arriba) Desviación típica del módulo de aceleración por ventanas de 2 s, coloreada según el estado y con las líneas de umbral. (Abajo) Tiempo total acumulado en cada estado.

Las figuras se han generado directamente a partir de los datos reales almacenados en InfluxDB y reproducen lo que muestran, en tiempo real, los paneles equivalentes de Grafana.

4.4 Ajuste de rendimiento y robustez del *pipeline*

Durante la puesta en marcha con datos reales —a máxima frecuencia y con muchos sensores activos a la vez— surgieron tres problemas de rendimiento e integridad de los datos cuyo diagnóstico forma parte de la experimentación y conviene documentar.

1) Cuello de botella en la escritura (latencia creciente). Con todos los sensores emitiendo, el broker entregaba varios miles de mensajes por segundo, pero la API los escribía en InfluxDB de forma **síncrona**, un `POST` por punto, lo que limitaba el ritmo a ~200 escrituras/s. La cola de ingesta crecía sin límite y la latencia del panel llegó a superar los 30 s. La solución fue sustituir la escritura síncrona por el **escriptor por lotes** del cliente de InfluxDB (`WriteOptions`), que acumula los puntos en memoria y los vuelca en bloque (cada 5 000 puntos o cada segundo). Con ello el consumo se equiparó a la producción y la frescura del panel bajó a menos de un segundo.

2) Colapso de muestras por falta de marca de tiempo (deduplicación). Tras activar los lotes se observó que solo se almacenaba ~1 muestra/s por sensor, pese a recibirse cientos. La causa: los puntos no llevaban marca de tiempo explícita, así que InfluxDB asignaba a todo el lote **un único instante** de ingesta; como los puntos de

un mismo *measurement* no tienen *tags* que los distinguan, todos los del lote colapsaban en una sola fila (InfluxDB identifica un punto por la tupla *measurement + tags + timestamp*). La solución fue **sellar cada mensaje con una marca de tiempo en nanosegundos** (`point.time(time.time_ns(), ...)`), que vuelve único cada punto y preserva la frecuencia de muestreo completa (verificado: de ~1 muestra/s se pasó a los cientos/s esperados).

3) Error de tipo en la consulta Flux (`cannot convert ... to float`). Los paneles de magnitud fallaban de forma intermitente con «*cannot convert argument type invalid to float*». El motivo es que en InfluxDB cada campo (`value_0`, `value_1`, `value_2`) es internamente una **serie temporal independiente**; en el borde más reciente de una consulta en vivo, el `pivot` puede componer una fila a la que aún no han llegado los tres ejes, dejando algún valor a `null` que hace fallar a `math.sqrt`. La solución fue **filtrar las filas incompletas** justo después del `pivot` (`filter(fn: (r) => exists r.value_0 and exists r.value_1 and exists r.value_2)`), de modo que la magnitud solo se calcula cuando están presentes las tres componentes.

Estos tres ajustes —escritura por lotes, marca de tiempo de alta resolución y filtrado de filas incompletas— son los que permiten que el sistema sostenga el flujo real de datos sin perder muestras ni acumular latencia, y se reflejan en el código final de `api/main.py` y en las consultas de los paneles.

5. Conclusiones y autoevaluación

Se ha desarrollado un prototipo **funcional y completo** de red de sensores que cubre las tres etapas planteadas: recogida (app Android propia), transmisión y almacenamiento (MQTT + FastAPI + InfluxDB) y análisis (Grafana). La arquitectura elegida —**publicación/suscripción asíncrona con MQTT** y **almacenamiento en serie temporal**— se ajusta de forma natural a las características de una WSN y permite añadir nodos o consumidores sin reescribir el sistema.

El principal valor del trabajo reside en la **fusión de conocimiento**: combinar varias señales sobre un eje temporal común para inferir información de alto nivel (la actividad del usuario), algo que ningún sensor aislado permitiría. El prototipo no se limita a visualizar esa fusión, sino que **clasifica la actividad de forma automática** (parado/caminando/corriendo) mediante umbrales sobre la variabilidad de la aceleración y contabiliza el tiempo en cada estado.

Como **líneas de mejora** se identifican: incorporar *tags* (`device`, `activity`) para enriquecer el modelo de datos y soportar varios dispositivos simultáneos; permitir configurar la frecuencia de muestreo desde la interfaz; sustituir el clasificador por umbrales —ya operativo— por un **modelo de aprendizaje automático** entrenado con los datos etiquetados, capaz de reconocer más actividades; y proteger el broker con autenticación y TLS para un despliegue real.

Autoevaluación

El equipo, formado por **Víctor Aguilar**, **Jaime Cazalla** y **Javier Dibo**, repartió el trabajo en tres frentes con colaboración cruzada y revisiones conjuntas:

- **Aplicación Android (productor)**: diseño de la interfaz, enumeración dinámica de sensores, *foreground service* con `WakeLock` y cliente MQTT (`MqttManager`).
- **Backend e infraestructura**: orquestación con Docker Compose (Mosquitto, InfluxDB, Grafana, FastAPI), la API puente MQTT→InfluxDB y la depuración del *pipeline* (sección 4.4).
- **Análisis y memoria**: diseño de los paneles y consultas Flux de Grafana (incluida la clasificación de actividad), la recogida etiquetada de datos y la redacción de este documento.

Grado de consecución. Se han cubierto **todos los objetivos** de la práctica —recogida, transmisión, almacenamiento y análisis con fusión de conocimiento— e incluso se ha ido más allá del mínimo exigido al añadir una **clasificación automática de la actividad** en tres estados y un panel de tiempos por estado.

Dificultades encontradas. Las principales fueron: (i) compatibilizar la cadena de compilación de Android (se fijó JDK 17 y Gradle 8.2, pues versiones más recientes daban problemas); (ii) los tres problemas de rendimiento e integridad del *pipeline* descritos en la sección 4.4 —latencia por escritura síncrona, colapso de muestras por falta de marca de tiempo y error de tipo en Flux—, cuyo diagnóstico fue la parte más instructiva del trabajo; y (iii) el ajuste de los umbrales del clasificador, que requirió capturar datos reales de cada actividad para calibrarlos.

En conjunto, la valoración del equipo es **muy positiva**: el resultado es un sistema completo, reproducible y defendible, y el proceso ha permitido comprender de primera mano los retos reales de una red de sensores.

6. Bibliografía

- Eclipse Mosquitto — *An open source MQTT broker*. <https://mosquitto.org/>
- OASIS — *MQTT Version 3.1.1 / 5.0 Specification*. <https://mqtt.org/>
- InfluxData — *InfluxDB 2 Documentation*. <https://docs.influxdata.com/influxdb/v2/>
- Grafana Labs — *Grafana Documentation*. <https://grafana.com/docs/>
- FastAPI — *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- Android Developers — *Sensors Overview*.
https://developer.android.com/guide/topics/sensors/sensors_overview
- Eclipse Paho — *MQTT Android/Java Client*. <https://www.eclipse.org/paho/>

7. Apéndices

A. Manual de despliegue del backend

Todo el código fuente del proyecto está disponible en el repositorio público de GitHub:

```
git clone https://github.com/jfdg01/seu-prac-3.git
cd seu-prac-3
```

A continuación, desde la carpeta `source-back-end` (que contiene `docker_compose.yaml`):

```
docker compose -f docker_compose.yaml up --build -d
```

Esto levanta los cuatro contenedores (Mosquitto :1883, InfluxDB :8086, Grafana :3000, API :8000). Se puede comprobar el estado con `docker ps` y los registros de la API con `docker logs sensores_api`. El detalle completo está en `source-back-end/docs/EJECUCIÓN.md`.

B. Configuración de la app móvil

1. Instalar el APK de `com.uja.sensores` (carpeta `source-android-app`) o compilarlo con Android Studio. El móvil y el ordenador deben estar en la **misma red WiFi**.
2. Introducir la **IP del ordenador** (obtenida con `ip a`), seleccionar los sensores deseados y pulsar *Iniciar transmisión*.

C. Configuración de Grafana

En <http://localhost:3000> (usuario `admin`, contraseña `admin`), añadir una fuente de datos **InfluxDB** con lenguaje **Flux**, URL `http://influxdb:8086`, organización `ujaen`, token `my-super-secret-auth-token` y bucket `sensores_bucket`. Los topics MQTT aparecerán como *measurements* al crear paneles.